Towards Connected Campus: A Secure, Role-Based, AI-Augmented Platform for Institutional Management

Dr. Mrs. Y Shree Vani
Dept. of Computer Science and Engineering
T. John Institute of Technology
Bengaluru, India
worksreevani@gmail.com

Vinay Babannavar
Dept. of Computer Science and Engineering
T. John Institute of Technology
Bengaluru, India
vinaybabannavar@gmail.com

Preetam Anil Kage
Dept. of Computer Science and Engineering
T. John Institute of Technology
Bengaluru, India
kagepreetam@gmail.com

Dilip T
Dept. of Computer Science and Engineering
T. John Institute of Technology
Bengaluru, India
dilipt122006@gmail.com

Prasad Reddy
Dept. of Computer Science and Engineering
T. John Institute of Technology
Bengaluru, India
prasadreddy.yerrasani@gmail.com

***Abstract*—Engineering colleges manage several processes that involve students, faculty members, administrators, security staff, and placement teams. In many cases, these processes are handled independently using paper forms, spreadsheets, messaging applications, or separate software systems. This makes it difficult to maintain a consistent flow of information between the people involved.**

This paper presents *Towards Connected Campus*, a role-based institutional management platform designed to bring several of these activities into a common system. The platform supports five roles: Student, Teacher, Head of Department (HOD), Security Guard, and Placement Officer. Its primary workflows include digital gate-pass management, student project tracking, placement assistance, academic calendar management, and real-time communication.

The gate-pass workflow replaces the traditional paper-based process with a sequential digital approval mechanism. A student's request is first reviewed by the class teacher and then by the HOD. After final approval, the system generates a QR code and a separate secret key that are used by security personnel to verify the pass at the campus exit. The platform also provides two AI-assisted features using the Google Gemini API. The first compares student profiles with placement job descriptions, while the second examines submitted GitHub repositories against students' claimed technical skills.

Security was considered as part of the system architecture rather than only as a frontend feature. Authentication uses signed JSON Web Tokens, passwords are stored using bcrypt hashing, protected routes use server-side role authorization, and the login endpoint is rate-limited. The paper describes the architecture, workflows, security measures, AI-assisted modules, testing approach, and current limitations of the platform.

***Index Terms*—Institutional Management Systems, Role-Based Access Control, Large Language Models, Digital Gate Pass, Placement Automation, Web Application Security, Real-Time Systems**

I. Introduction

Engineering colleges handle a large number of activities involving students, faculty members, department administrators, security personnel, and placement teams. Many of these activities are still managed through separate processes. For example, a student who needs to leave the campus during working hours may have to obtain approval from a class teacher and the Head of Department before presenting the approved document to security staff. Placement teams, meanwhile, may have to compare job descriptions with a large number of student resumes manually.

These processes are not necessarily difficult because of the number of individual steps involved. The larger problem is that the information created in one step is often disconnected from the next step. A gate-pass request, for example, may begin with a student, continue with a teacher and HOD, and finally reach the security guard. When these activities depend

on paper or independently maintained records, information has to be transferred manually between the participants.

A similar problem occurs during placement activities. A student's technical skills and project experience may be stored in a resume or profile, while the actual implementation of those projects remains outside the placement system. Consequently, a placement officer has limited automated support for comparing a student's stated skills with the work they have actually completed.

Towards Connected Campus was developed to address these problems through a common web platform. Instead of creating separate applications for individual activities, the system maintains a shared backend and database while providing different interfaces according to the user's role. A gate-pass created by a student remains the same record when it is reviewed by a teacher, approved by an HOD, and verified by a security guard.

The same shared-data approach is used for placement-related activities. Student profiles are maintained within the platform and can be used as input to AI-assisted matching and project verification features.

The remainder of this paper is organized as follows. Section II discusses existing approaches and the gap addressed by the proposed platform. Section III presents the system architecture. Section IV describes the cross-role workflow. Section V explains the major modules and security design. Section VI describes the testing approach. Section VII discusses design observations and limitations. Section VIII summarizes the technology stack. Section IX concludes the paper and describes possible future improvements.

II. Related Work

Campus management applications are commonly developed around specific institutional requirements. Existing approaches can be grouped into general college ERP systems, dedicated gate-pass applications, and placement or resume-screening tools.

A. Generic College ERP Systems

College ERP systems normally combine administrative functions such as attendance, fee management, examination records, and student information. Their broad scope makes them useful for maintaining institutional records. However, specialized workflows such as a multi-level gate-pass process or AI-assisted placement analysis may not receive the same level of integration as the core administrative functions.

The proposed platform takes a narrower approach. Instead of attempting to replace every college ERP function, it concentrates on workflows involving student movement, academic interaction, communication, project information, and placement activities.

B. Gate-Pass and Visitor Management Systems

Dedicated gate-pass and visitor-management applications are designed to replace paper-based entry and exit procedures. QR codes are often used to make verification faster at a physical checkpoint.

Such systems can simplify the approval process, but they generally focus on entry and exit management alone. The gate-pass information may therefore remain separate from student project records, placement information, and academic activities.

In the proposed platform, the gate-pass record belongs to the same student record system used by the other institutional modules. This allows the approval workflow to operate across student, teacher, HOD, and security roles.

C. Placement and Resume-Screening Tools

Resume-screening systems can reduce the amount of manual work required when a placement team receives a large number of applications. Traditional approaches often rely on keywords or structured fields to compare resumes with job descriptions.

Keyword-based matching can be useful for identifying explicit terms, but it may not capture the context of a project description. For example, a student may describe a project using different words from those used in a job description while still having relevant experience.

The proposed placement module therefore uses a large language model to compare structured student information with a job description. The generated results are intended to assist the placement officer rather than replace the officer's decision.

D. Gap Addressed by This Work

The main focus of this work is the integration of several cross-role activities within one platform. The system combines a digital approval workflow, placement assistance, project tracking, skill verification, real-time communication, and server-side role-based authorization.

Rather than treating these activities as independent applications, the platform maintains a shared institutional data layer. This provides a common point through which the participating roles can interact with the same records.

III. System Architecture

The platform follows a client-server architecture consisting of a web-based frontend, an Express backend, a relational database, real-time communication services, and an external large language model API.

The major components and their relationships are shown in Fig. 1.

A. Frontend Layer

The frontend is implemented using React 19 and Vite. Tailwind CSS is used for styling, while Framer Motion provides interface transitions. The application also uses `@use-gesture/react` for touch interactions such as drag-to-dismiss behavior on mobile interfaces.

After authentication, the user is directed to the interface corresponding to the assigned role. The five supported roles are Student, Teacher, HOD, Security Guard, and Placement Officer.

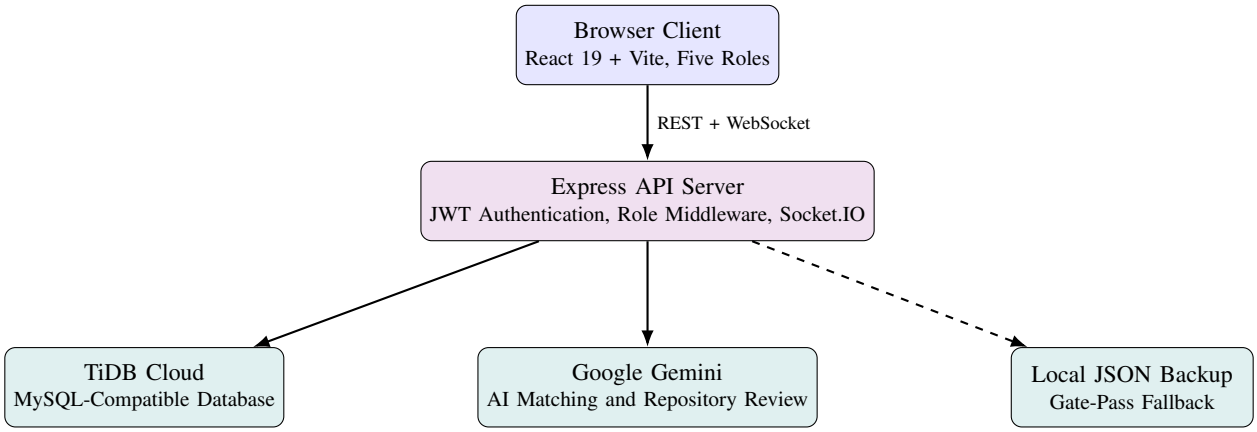


Fig. 1. System architecture of the proposed platform. The browser client communicates with the Express server through REST and WebSocket connections. The backend stores persistent information in TiDB Cloud, uses Google Gemini for the two AI-assisted operations, and maintains a local JSON fallback for gate-pass records.

The frontend is responsible for presenting role-specific operations, but it does not serve as the final authorization layer. Access to protected operations is checked again by the backend.

B. Backend Layer

The backend is implemented using Node.js and Express 5. REST endpoints are organized around the major application domains, including authentication, student information, gate passes, placement drives, registrations, and AI-assisted operations.

Socket.IO runs alongside the REST API and provides real-time communication for the Campus Connect chat module.

C. Authentication and Authorization

Protected endpoints require a valid signed JSON Web Token. The token issued during login contains the authenticated user's role and has a 12-hour expiry.

After the token is verified, server-side middleware checks whether the role associated with the token is allowed to perform the requested operation. This prevents the application from relying only on frontend restrictions.

The login endpoint is also rate-limited to 15 requests within a 15-minute window per client. Cross-Origin Resource Sharing is restricted to the configured application origin.

These measures were introduced as part of the security hardening of the application.

D. Data Layer

The persistent data layer uses TiDB Cloud, which provides a MySQL-compatible relational database. The platform stores student records, gate-pass information, placement drives, registrations, and chat messages in the database.

Database queries are parameterized to reduce the risk of SQL injection.

For gate-pass records, a local JSON file is also maintained as a fallback write path. This is intended to provide a temporary alternative if database connectivity becomes unavailable during an active gate-pass operation.

The client uses browser LocalStorage to retain the session token and support continuity across page reloads.

E. AI Integration Layer

The platform uses the Google Gemini API for two application features. The first feature compares student profiles with placement job descriptions. The second examines a student's submitted GitHub repository and compares the repository contents with the technical skills claimed by the student.

The system does not train a new machine-learning model. Instead, it uses a pretrained large language model through structured prompts. Relevant student, job-description, and repository information is provided as context when an AI-assisted operation is requested.

IV. Cross-Role Platform Workflow

The platform is organized around five user roles. Each role authenticates through the same backend but receives access to a different set of operations.

The gate-pass workflow connects four roles in sequence:

$$\text{Student} \rightarrow \text{Teacher} \rightarrow \text{HOD} \rightarrow \text{Security Guard}.$$

The Placement Officer works directly with student and placement information and does not participate in the gate-pass approval chain.

Fig. 2 illustrates the relationship between the roles and the shared backend.

Table I summarizes the main server-enforced permissions for each role.

V. Core Modules

A. Digital Gate Pass

The digital gate-pass module replaces the paper approval process with a sequence of controlled steps.

First, the student submits a gate-pass request through the application. The request is then forwarded to the student's class

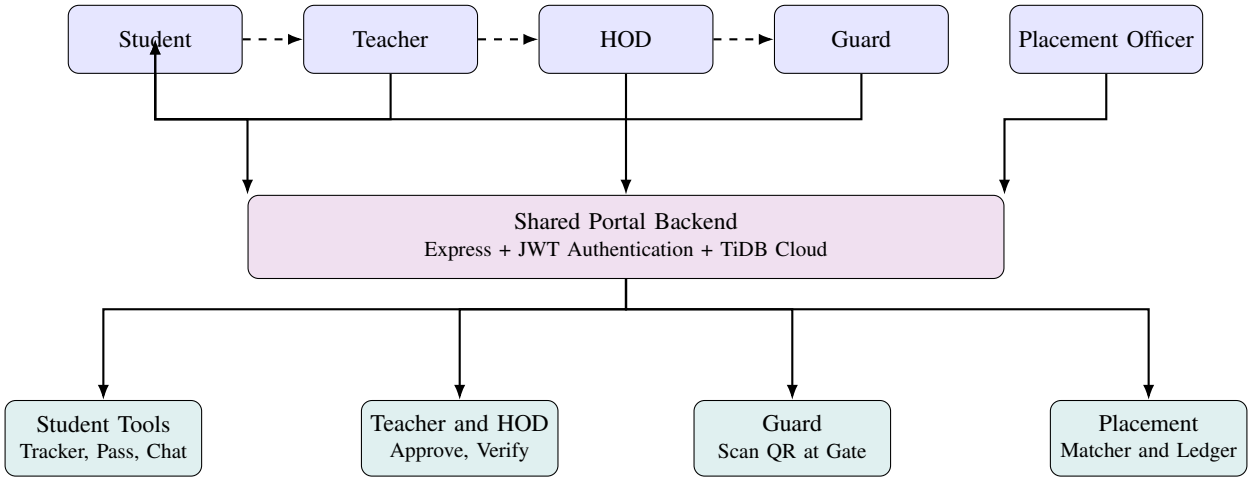


Fig. 2. Cross-role workflow of the platform. Dashed arrows represent the gate-pass approval chain from Student to Teacher to HOD and finally to the Security Guard. All five roles authenticate through the shared backend.

TABLE I
ROLE PERMISSION MATRIX

Role	Server-Enforced Permissions
Student	Create personal gate-pass requests; view own profile and projects; view placement eligibility; use chat and calendar.
Teacher	Approve or reject gate-pass requests at the first approval level; view assigned students.
HOD	Provide final gate-pass approval; trigger QR and secret-key generation; view department-level exit information.
Guard	Scan a gate-pass QR code; enter the associated secret key; validate and record a student's exit.
Placement Officer	Run AI-assisted job-description matching; manage placement drives; view student registrations.

teacher. After teacher approval, it is made available to the HOD for the second level of approval.

When the HOD approves the request, the system generates a QR code along with a numeric secret key. The QR code is provided to the student, while the secret key is not displayed to the student or teacher.

At the campus gate, the security guard scans the QR code and enters the secret key separately. The backend validates both pieces of information before recording the student's exit.

Keeping the secret key separate from the QR code adds an additional verification step. A public verification page is also available so that a person such as a parent can check whether a gate pass is genuine without accessing the protected application.

B. AI-Assisted Placement Matching

The JD Matcher is designed to assist the Placement Officer when evaluating students for a particular job description.

The officer provides a job description and can optionally provide a set of resumes. The system prepares relevant student profile information and job-description text and sends the structured context to the Gemini API.

The generated result contains a candidate ranking together with supporting reasoning. The result is presented to the Placement Officer for review rather than being treated as an automatic hiring decision.

This approach allows the system to consider the meaning of project descriptions instead of depending entirely on exact keyword matches.

C. Project Tracker and Skill Verification

The Project Tracker allows a student to submit a GitHub repository associated with a project.

The backend obtains information about the repository's structure and source files and provides relevant content to the Gemini-based verification process. The model is asked to compare the student's claimed technical skills with evidence present in the repository.

For example, if a student claims experience with React, the system can look for React-related implementation within the submitted project rather than relying only on the word "React" appearing in a resume.

The resulting output is intended as supporting evidence. It does not replace a technical interview or a manual review of the student's work.

D. Security Hardening

Security became an important part of the development process after reviewing the initial authentication implementation.

The earlier implementation relied partly on client-provided role information and included fallback login credentials. These design choices could have allowed unauthorized access if a malicious client modified the request.

The authentication mechanism was subsequently changed to use bcrypt-hashed passwords and server-issued JWTs. The role used for authorization is taken from the verified token rather than from a client-supplied role value.

Two middleware functions, `requireAuth` and `requireRole`, are used to protect routes. For example, a Student token cannot be used to access a Teacher- or HOD-only gate-pass approval endpoint.

The login endpoint is rate-limited, and CORS is restricted to the configured frontend origin.

These changes reinforce the principle that role-based security must be enforced at the server. Restricting buttons or pages in the frontend alone does not prevent a user from sending a direct request to an API endpoint.

E. Academic Calendar and Real-Time Communication

The academic calendar provides a common place for displaying semester timelines, internal assessment schedules, and institutional events.

The Campus Connect module uses Socket.IO to provide real-time communication. It supports messaging as well as online presence and typing indicators. The purpose of the module is to provide a platform-managed communication channel for interactions that may otherwise take place through informal messaging groups.

VI. Testing Approach

Testing focused on the workflows that involve multiple roles and security boundaries. Rather than testing only individual functions, the main scenarios were exercised from the perspective of the users who participate in each workflow.

A. Gate-Pass Workflow Testing

The complete gate-pass sequence was tested from request creation to exit verification.

The following stages were checked:

1) Student submits a gate-pass request.
2) Teacher reviews and approves or rejects the request.
3) HOD provides the final approval.
4) QR code and secret key are generated after HOD approval.
5) Student receives the QR code without receiving the secret key.
6) Guard scans the QR code and validates the secret key.

The visibility of information at each stage was also checked against the permissions defined in Table I.

B. Authorization Boundary Testing

Authorization was tested by sending requests directly to protected API endpoints instead of relying only on the frontend.

Tokens associated with different roles were used to verify that a Student token could not access Teacher- or HOD-only operations.

The authentication process was also tested after removing the previous fallback credentials to confirm that those credentials could no longer be used to access accounts.

C. Cross-Role Data Consistency

A gate-pass request created by a Student was followed through the Teacher and HOD interfaces to confirm that all roles were working with the same underlying database record.

This test was important because the platform is designed around a single source of truth rather than independently synchronized copies of the same record.

D. Testing Observation

The testing process showed that functional correctness alone is not sufficient for a multi-role institutional application. The same operation must also be checked from the perspective of unauthorized users.

For this reason, direct API testing was included alongside normal frontend-based workflow testing.

VII. Discussion

A. Shared Data Across Roles

A central design decision in the platform is that important institutional records are maintained once and accessed by multiple roles according to their permissions.

For example, a gate-pass request is not copied into separate teacher, HOD, and security records. The same request moves through different states as it passes through the approval process.

This approach reduces the possibility of conflicting copies of the same information and makes the workflow easier to trace.

B. Security as an Early Design Requirement

The authentication changes made during development highlighted the cost of adding security after application features have already been implemented.

Once multiple API routes and user roles exist, changing the authorization model requires checking each protected operation.

For institutional systems that store student information, authentication and authorization should therefore be considered from the beginning of development rather than added after the functional features are complete.

C. Limitations of AI-Assisted Features

The AI-assisted placement and skill-verification modules have limitations.

Both features depend on the information supplied to the language model. If a student's profile is incomplete, a project repository is private or unavailable, or the submitted information does not represent the student's actual work, the generated result may not accurately describe the candidate.

The AI output should therefore be treated as decision support. Final decisions should remain with the Placement Officer or other responsible institutional staff.

D. Deployment Considerations

The current backend operates as a standalone Node.js and Express process, while the frontend is served through Vite.

This structure is suitable for development and can also support a single-server or on-campus deployment. However, additional work is required before operating the platform at a larger institutional scale.

In particular, the system would benefit from load testing under realistic concurrent usage and further deployment work if a serverless hosting architecture is selected.

VIII. Technology Stack

Table II summarizes the major technologies used in the platform.

TABLE II
Technology Stack

Layer	Technology
Frontend	React 19, Vite, Tailwind CSS, Framer Motion, `@use-gesture/react`
Backend	Node.js, Express 5, Socket.IO
Authentication and Security	JSON Web Tokens, bcrypt password hashing, `express-rate-limit`
Database	TiDB Cloud, MySQL-compatible distributed database
AI	Google Gemini API
Client Storage	Browser LocalStorage

IX. Conclusion

Towards Connected Campus brings several campus activities into a single role-based platform. The current implementation connects students, teachers, HODs, security personnel, and placement officers through a shared backend and database.

The digital gate-pass workflow demonstrates how a process that normally depends on paper approvals can be converted into a controlled digital sequence. Teacher and HOD approvals are recorded within the system, while QR and secret-key verification provides a mechanism for checking the approved pass at the campus gate.

The placement-related modules extend the platform beyond administrative workflows. AI-assisted job-description matching provides placement officers with additional information when reviewing candidates, while repository-based skill verification provides another source of evidence about a student's technical experience.

The security work carried out during development also demonstrates the importance of server-side authorization. Authentication, password protection, role validation, rate limiting, and CORS configuration are treated as application-level requirements rather than frontend-only restrictions.

The current platform still requires further evaluation before large-scale institutional deployment. Future work includes an activity and achievement verification module for certifications, hackathons, and community-service activities; further deployment optimization; load testing under realistic campus traffic; and a broader evaluation involving students, faculty members, security staff, and placement personnel.

References

[1] Meta Platforms, Inc., "React Documentation," Available: https://react.dev

[2] OpenJS Foundation, "Node.js Documentation," Available: https://nodejs.org

[3] Express.js, "Express: Fast, Unopinionated, Minimalist Web Framework for Node.js," Available: https://expressjs.com

[4] Socket.IO, "Socket.IO Documentation," Available: https://socket.io/docs

[5] PingCAP, "TiDB Cloud Documentation," Available: https://docs.pingcap.com/tidbcloud/

[6] Google, "Gemini API Documentation," Available: https://ai.google.dev/docs

[7] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015.

[8] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in *Proceedings of the USENIX Annual Technical Conference*, 1999.

[9] OWASP Foundation, "OWASP Top Ten," Available: https://owasp.org/www-project-top-ten/

[10] V. Babannavar *et al.*, "Towards a Connected Campus: Design and Evaluation of an AI-Augmented Institutional Management Platform," *International Journal of Scientific Research and Engineering Development (IJSRED)*, vol. 9, no. 3, May–June 2026.